Resilient Anomaly-Aware Building Access Control

D7065E, Embedded Intelligence at the Edge — Architecture Document

Stefan Hoglund

Contents

1 Summary **2**

2 Use case and building context **2**

3 Requirements **3**

4 Architecture **4**
4.1 Context diagram (C4 Level 1) 4
4.2 Container diagram (C4 Level 2) 5
4.3 Component diagram (C4 Level 3) 7
4.4 Design decisions and alternatives 8

5 Interfaces and communication **9**

6 Simulation **10**

7 Autonomous decision-making and data pipeline **11**
7.1 The autonomous service 11
7.2 The data pipeline 12

8 Behaviour **13**
8.1 Key scenario: successful passage (dynamic diagram) 13
8.2 Door-control lifecycle (state diagram) 14

9 Deployment **15**

10 Test plan and verification **16**

11 Evaluation **17**
11.1 Nominal correctness 17
11.2 Latency (REQ-PERF-001) 17
11.3 Failure / recovery 18
11.4 Anomaly policy behaviour 18
11.5 Stress / breaking-point experiment 19

12 Dashboard **20**

13 Risks, limitations, and future work **21**

14 Conclusion **22**

1 Summary

This project implements a distributed cyber-physical access-control system for a single controlled boundary in a BuildSim building: a simulated population presents credentials at a card reader, an autonomous Decision Service evaluates authorization and behavioral anomaly as two independent signals, a Door Actuator carries out the resulting lock/unlock commands against BuildSim, and independent sensor processes observe the physical consequences and feed them back into the next decision. Eight Go processes plus the course-provided BuildSim run as either native binaries or Docker containers; an append-only evidence log retains every observation, decision, command, and outcome for replay and evaluation; a dashboard shows live system state.

The guiding principle threaded through every design decision is *domain separation under observable evidence*: authorization answers "is this permitted", anomaly detection answers "is this usual", and neither is allowed to silently stand in for the other anywhere in the code – not as an optimization, but because the course's own learning objectives (architectural, systems, and critical thinking) are best demonstrated by a system whose separations are structural, testable, and falsifiable, not just asserted in prose.

Field	Value
Project title	Resilient Anomaly-Aware Building Access Control
Student	Stefan Hoglund
Use case	Single-boundary building access control with autonomous authorization and anomaly detection
Repository	`applied-ai-edge-computing` (https://github.com/stefanhoglund/building-access-control; see `README.md`)
Version / date	Final submission pass, October 2026

Scope and evolution. The system settled on a materially smaller physical footprint than a typical multi-room ventilation use case would require (one controlled door, not many rooms with continuous physical quantities), and used that headroom instead on distributed-systems depth: fail-closed behavior under a killed Decision Service, unverified-vs- confirmed actuation under a killed BuildSim/Actuator, a genuine stress/breaking-point experiment, and a full Docker Compose deployment alongside the native one. The autonomous component stayed deliberately rule-based (authorization precedence plus a two-factor anomaly scorer) rather than adopting a learned model: with one controlled door there is no data-quantity or feature-diversity justification for machine learning, and a deterministic baseline is what makes the decision-policy invariants (Section 7) checkable by a unit test rather than only by inspection. This is consistent with the course's own framing that sophistication is not the grading criterion (grading criteria, "Expectations and Scope").

2 Use case and building context

The use case is access control at one controlled boundary between a public zone and an employee zone in a BuildSim office floor: a reader at the boundary accepts a presented credential, and the system must decide, autonomously and in bounded time, whether to unlock the door — while also noticing when an otherwise-authorized presentation looks unusual enough to flag for a human to review later. BuildSim supplies the shared ground truth for the physical world this project does not own: building topology, the door's lock state and open/closed position, and a 3D viewer. It is explicitly *not* the authorization engine, the anomaly detector, the occupant-behavior simulator, or the historical record (CLAUDE.md §4) — this project supplies all four of those, around BuildSim's physical substrate.

The tension that makes the decision non-trivial is the same one real access-control systems face: authorization alone is necessary but not sufficient for security assurance. A credential can be valid and the access still worth flagging — an employee badging in at 3 a.m., the same badge presented three times in five seconds. Conflating "permitted" with "normal" would either over-trigger alerts on every legitimate late night, or silently miss exactly the behavior worth noticing. The project therefore treats authorization and anomaly as two independently computed signals that are only combined at the very end, by an explicit decision policy, never earlier (Section 7).

The cyber-physical feedback loop this project closes is: a simulated person's intent (Occupant & Movement Simulator) becomes a sensor observation (Card Reader) → a validated, stored event (Event/Data Service) → an autonomous decision (Decision Service) → a physical actuator command (Door Actuator) → an observable BuildSim state change (lock state, door position) → the Simulator causing the physical crossing conditioned on that state → independent sensors (DoorStateSensor, OccupancySensor) observing the *new* state → fed back into evidence the next decision (and the dashboard) can use. Section 8 walks this loop for one concrete scenario end-to-end.

3 Requirements

Requirements are grouped by prefix (`docs/requirements.md`, 54 IDs total) and verified in `docs/traceability-matrix.md`, which is this project's single authoritative requirement-status table (see Section 10). The table below is a representative selection by group; the full table, with implementing component, test, evidence, status, and a one-sentence rationale for all 54 IDs, is in that document.

ID	Requirement (concise)	Status
REQ-FUNC-001/002	Deny/allow an access attempt per the authorization result	PASS
REQ-FUNC-003/004	Relock on no-open timeout / after a completed passage	PASS
REQ-AUTH-001–008	Card/user/role/permission model, deterministic precedence (ADR-001)	5 PASS, 2 PARTIAL, 1 UNIMPLEMENTED
REQ-TOPO-001–004	Zones, controlled boundary, reader/actuator/door-state binding, level-qualified room IDs	3 PASS, 1 PARTIAL
REQ-SIM-001–006	Configured population, named seeded scenarios, occupancy truth, CPS feedback	4 PASS, 2 PARTIAL
REQ-DEC-001–007	Autonomous decision, authorization/anomaly separation, full traceability	7 PASS
REQ-DIST-001–007	Independent restartable processes, documented interfaces, bounded timeouts, dedup	5 PASS, 2 UNIMPLEMENTED
REQ-DATA-001–006	Data pipeline, restart survival, required fields, duplicate/late/malformed handling	3 PASS, 1 PARTIAL, 2 UNIMPLEMENTED
REQ-RES-001–005	Fail-closed, unverified actuation, staleness, restart recovery	3 PASS, 2 UNIMPLEMENTED
REQ-VIS-001	Dashboard visualization	PASS
REQ-BS-001–003	BuildSim authoritative role, independence of history, observable feedback	3 PASS
REQ-PERF-001	p95 decision latency within threshold	PASS (measured, Section 11)
REQ-TEST-001/002	Reproducible scenario ID + seed; full traceability	2 PASS

How verification works. Each requirement names the component that owns it, the automated test or manual experiment that exercises it, and the evidence directory or log where that is recorded – `docs/traceability-matrix.md` Part B is this table in full, and Part A is the course's own mandatory pass/fail checklist, verified separately. Of 54 internal requirements, **41 are PASS,**

6 PARTIAL, 7 UNIMPLEMENTED – the PARTIAL/UNIMPLEMENTED set is discussed honestly in Section 13, not hidden here.

Non-functional requirements are the `REQ-DIST`, `REQ-DATA`, `REQ-RES`, and `REQ-PERF` groups (timeouts, restart behavior, data retention, latency); functional requirements are `REQ-FUNC`, `REQ-AUTH`, `REQ-TOPO`, `REQ-SIM`, `REQ-DEC`, `REQ-VIS`, `REQ-BS`. `REQ-TEST-*` and the mandatory course checklist (reproduced as Part A of the traceability matrix) are kept explicitly distinct from both: they are evaluation-methodology and gate requirements, not system behavior requirements.

4 Architecture

4.1 Context diagram (C4 Level 1)

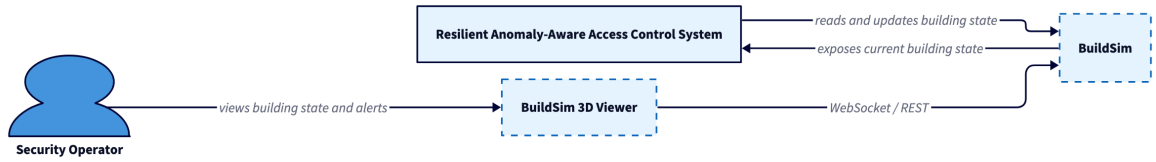


Figure 1: System context: the Resilient Anomaly-Aware Building Access Control system, its one human role (Security Operator), and the one external system it depends on (BuildSim).

The system is a single logical unit from the outside: a Security Operator views its dashboard; it reads from and writes to BuildSim over REST. Everything else – the simulated population, the authorization policy, the anomaly baseline, and the evidence store – is internal. No other external system exists; there is no cloud backend, no separate identity provider, no message broker. This is a deliberate consequence of the course's own scope guidance ("you are not expected to implement ... large-scale cloud-native infrastructure") rather than an omission.

4.2 Container diagram (C4 Level 2)

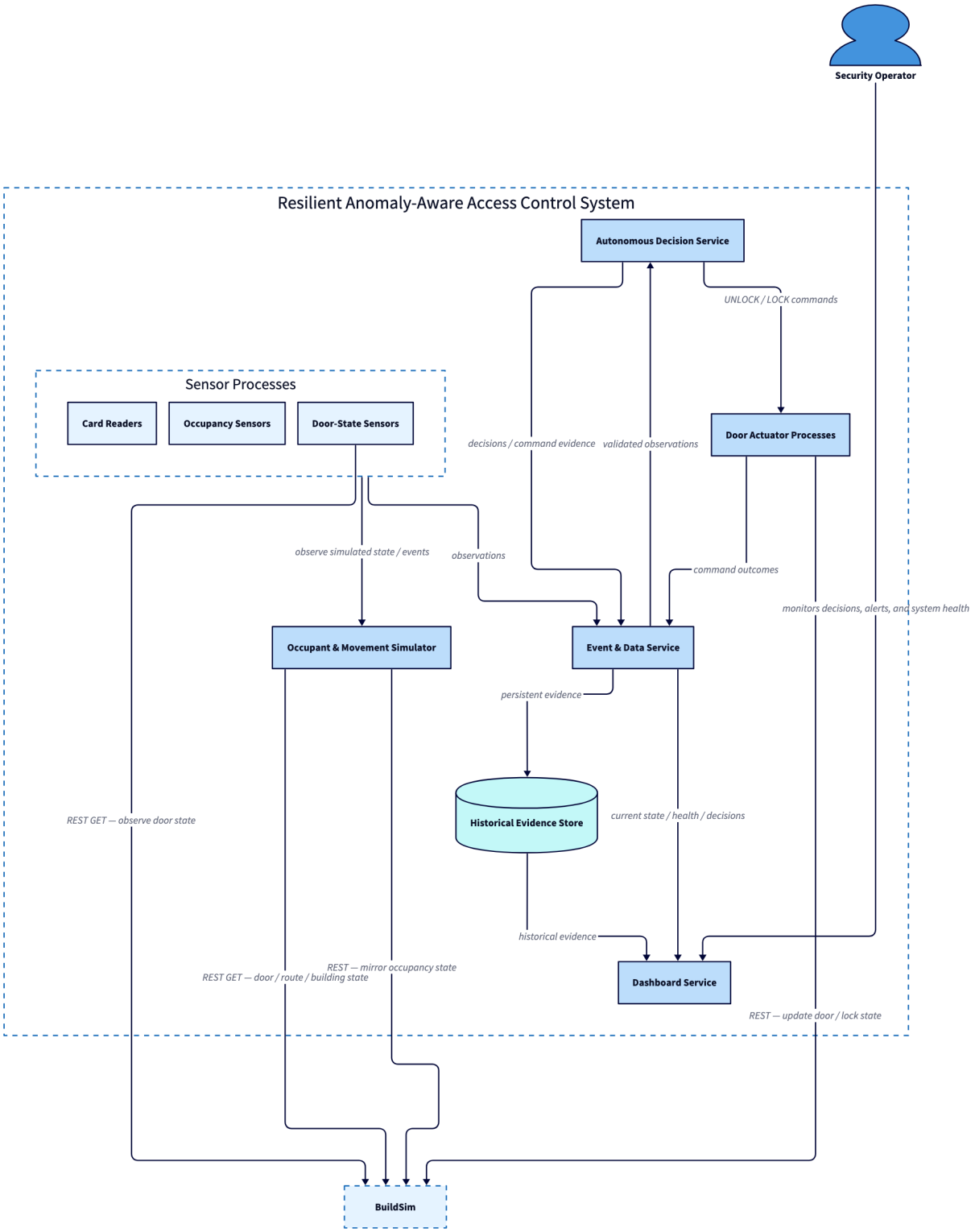


Figure 2: Container diagram: the eight independently deployable project processes and their communication paths, plus BuildSim.

Opening the system into its separately-running parts: the Occupant & Movement Simulator (owns model time and occupancy/door-position truth); three independent sensor processes (Card Reader, OccupancySensor, DoorStateSensor); the Event/Data Service (validates and retains

evidence, and sits on the control path – Section 5); the Decision Service (authorization, anomaly, policy, runtime assurance); the Door Actuator (issues and verifies physical commands); and the Dashboard (reads the evidence file directly). Every arrow is a real REST call or a direct file read, documented precisely in `docs/data-contracts.md`. The system is split this way because each box is independently the unit of restart, failure, and deployment the course's mandatory pass/fail criterion #1 requires – not because eight is an aesthetically pleasing number of services. Section 4.4 below names the alternatives actually considered and rejected for each major boundary.

4.3 Component diagram (C4 Level 3)

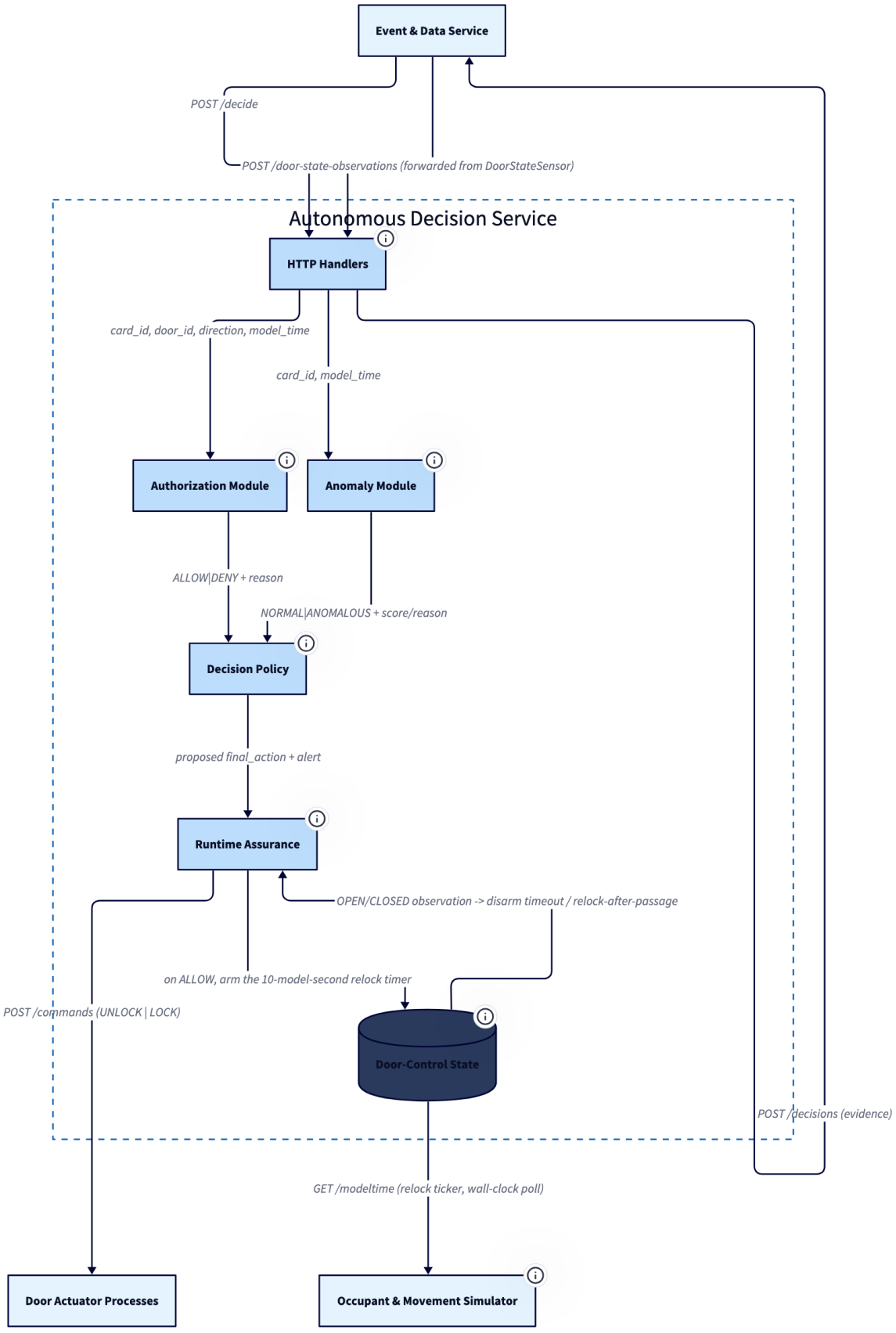


Figure 3: Component diagram: the Decision Service's internal module structure.

The Decision Service is the one container worth opening further, because its internal separation is itself a graded design decision (ADR-002). Four logically distinct modules run inside one process: **authz.go** (deterministic precedence authorization, ADR-001), **anomaly.go** (the two-factor deterministic anomaly scorer), **server.go**'s decision policy (combines the two, and *only* the two, into `final_action`), and the per-door control/runtime-assurance state (relock timers, unlock bookkeeping). The module boundary is enforced in code, not just in the diagram: `authorize()` and `anomalyTracker.score()` never read each other's return value – this is why REQ-DEC-002/003 are PASS by direct test, not by convention.

4.4 Design decisions and alternatives

Decision	Alternatives considered	Why this one
One Autonomous Decision Service process containing four logically separate modules (ADR-002)	(a) one network microservice per module (authorization/anomaly/policy/ runtime-assurance each independently deployable); (b) all decision logic folded directly into the Event/Data Service	(a) would add three more network hops, timeout surfaces, and deployment units for zero required-by-the-course benefit – the mandatory criterion asks for *an* independently deployable autonomous component, not one per internal function. (b) would blur the Event/Data Service's own single responsibility (ADR-005) and make the authorization/anomaly separation a code-review convention instead of a process boundary. The chosen option keeps the separation structurally enforced in code while avoiding unjustified network calls.
Sensor observations route through the Event/Data Service before reaching the Decision Service, not directly sensor-to-decision (ADR-005)	(a) sensors call the Decision Service directly, with the Event/Data Service only receiving a copy for storage; (b) the chosen path, Event/Data Service as the sole gate	(a) creates two sources of truth for "was this observation accepted" and makes validation/dedup logic (even the parts not yet built, REQ-DATA-004/ 005) have to live in two places or be skipped. (b) gives the system one place malformed, duplicate, or late data is handled, and makes the Historical Evidence Store's "sole writer" invariant (ADR-005) trivial to state and check. The cost is that the Event/Data Service is now on the critical access-control path – named explicitly as a trade-off in ADR-005 and revisited in Section 13.
REST over HTTP for every inter-process call, not a message broker	MQTT/a message broker (explicitly suggested by the course materials as a comparison point)	With nine processes, mostly request/response in shape ("decide this", "confirm this command"), and no requirement for pub/sub fan-out or offline message buffering, a broker adds an extra piece of infrastructure, deployment surface, and failure mode without an offsetting benefit. REST with bounded timeouts (`internal/httpx`, 2s default, REQ-DIST-004) is precise enough to document per-endpoint (`docs/data-contracts.md`) and simple enough that every one of this project's fault experiments could inject a failure at a well-defined point.

Decision	Alternatives considered	Why this one
Append-only JSONL file as the Historical Evidence Store, not a database	PostgreSQL/SQLite; an embedded key-value store	No query pattern in this project needs indexing, joins, or concurrent writers (ADR-005: Event/Data Service is the sole writer) – a database would add an operational dependency and a schema-migration surface for zero required capability. A plain file is trivially restart-survivable (REQ-DATA-002, directly demonstrated) and trivially inspectable by a human or a test (`evidence.ReadAll`).
Deterministic rule-based anomaly scoring, not a learned model	A trained classifier (e.g., logistic regression or an LLM-based judge) on access-time/repeat-attempt features	With one controlled door and a handful of simulated people, there is no realistic quantity or diversity of training data to justify a learned model, and a rule-based baseline makes the core safety invariant (anomaly can never grant access, REQ-DEC-003) a one-line structural guarantee checkable by `go test`, not a property that would need to be re-verified after every retrain. The course's own grading philosophy states explicitly that more sophisticated AI does not by itself earn a higher grade.
Docker Compose, one image shared by all eight project services, plus BuildSim's own (unmodified) course-provided image	Kubernetes; one Dockerfile/image per service	Nine containers that mostly talk over plain REST on one host do not need an orchestrator with its own control plane – the course explicitly discourages this (§3: "do not introduce Kubernetes ... unless explicitly required by an experiment"). Building eight separate images for eight binaries that share 100% of their source tree would only multiply build time and image count for no operational benefit; one multi-stage image with per-service `command:` overrides keeps the image count low while every container still restarts independently (verified, Section 9).

5 Interfaces and communication

Every inter-process interface is REST over HTTP/JSON, fully specified in `docs/data-contracts.md`: endpoint, method, request/response schema, and an example payload for each. Representative examples:

- `POST /observations` (Event/Data Service) – `AccessAttemptObservation` (`event_id`, `source_id`, `model_time`, `source_time`, `receive_time`, `schema_version`, `card_id`, `door_id`, `direction`) – the Event/Data Service stamps `receive_time`, persists the record, and forwards it to `POST /decide`.
- `POST /decide` (Decision Service) – the same observation; response is `AccessDecision` (`decision_id`, `authorization_result/reason`, `anomaly_result/reason/ score`, `final_action`, `alert`, `policy_version`).
- `POST /commands` (Door Actuator) – `ActuatorCommand` (`command_id`, `decision_id`, `action`, `reason`); response path produces `ActuatorOutcome` (`result` ∈ {`CONFIRMED`, `UNVERIFIED`}, `observed_state`).
- `POST /scenario/run` (Simulator) – `{scenario, seed, count?, interval_ms?}`; blocks until every presentation the named scenario specifies has been enqueued, returns `{run_id,`

`scenario, seed, model_start, steps_executed, card_ids}` (Section 6).

Every network call has a bounded timeout (`internal/httpx`, `DefaultTimeout = 2s`, REQ-DIST-004) – no inter-process call can block indefinitely, which is what makes the fail-closed behavior under a killed Decision Service (Section 11) a bounded-time property rather than a hang.

Control-path design (ADR-005). Observations are routed Sensor → Event/Data Service → Decision Service, not Sensor → Decision Service directly. This puts the Event/Data Service on the critical access-control path – a deliberate trade-off (Section 4.4): it buys one validation/evidence boundary instead of two, at the cost of one additional process the decision latency now depends on. Decision latency was measured specifically with this path included (Section 11) and remains far below threshold.

Evidence flow. The Event/Data Service is the Historical Evidence Store's sole writer (ADR-005); every observation, decision, command, and outcome is appended to `data/evidence.jsonl` as it is produced. The Dashboard and every evaluation tool read this file directly and re-read it fresh on every request – there is no separate read API, by design (one less interface to keep consistent with the write path).

BuildSim adapter. BuildSim's `PUT /api/doors` and `PUT /api/occupancy` both wholesale-replace their entire collections (confirmed from BuildSim's own source, not assumed). The `internal/buildsim.HTTPClient` adapter hides this behind a GET-modify-PUT pattern: `SetDoorState`/`SetDoorPosition` read the full current door list, mutate only the targeted door's field (preserving every other door's existing wire data, including fields this project does not otherwise use, such as `entry_node_id`), and PUT the whole collection back. The same pattern applies to room occupancy mirroring (`SetRoomOccupancy`), which also round-trips BuildSim's `aliens` field even though this project never populates it, purely so an unrelated field is never silently dropped on a PUT.

Model time vs. wall-clock time. The Occupant & Movement Simulator is the sole owner of model time (ADR-003); every other process either queries it (`GET /modeltime`) or receives it stamped on an observation, never infers it from its own clock. Model time governs permission windows, scenario timing, and the anomaly scorer's time-of-day factor; wall-clock time governs network timeouts, process health, and every latency/recovery-time measurement in Section 11. The two are never mixed implicitly – every timer in the codebase states which clock it uses in its own comment (e.g., the relock-timeout ticker polls on a wall-clock cadence but compares against model time).

6 Simulation

The Occupant & Movement Simulator owns model time and simulated occupant/occupancy truth (ADR-003, ADR-006). It is deliberately small: exactly one controlled door, and a configured population of eight simulated people (`config/population.json`) spanning every role the authorization configuration defines (employee, facilities, manager, visitor, student) – behavioral diversity, not population size, is the design goal, consistent with the course's own framing that a simple, well-justified system scores higher than a technically ambitious one that cannot be clearly explained.

Eight named, seeded, reproducible scenarios (`config/scenarios.json`, `docs/scenarios.md`) are invoked via `POST /scenario/run`: `normal-day` (a seeded random subset/order/spacing of the population), `authorized-passage`, `unauthorized-access`, `after-hours-access`, `rapid-repeat-access`, `burst-access`, and two scenarios (`decision-service-failure`, `actuator-or-buildsim-failure`) whose role is to drive a single reproducible presentation for an operator to pair with a process kill/restart – the scenario itself never performs process control, since the Simulator must not reach outside its own domain to manage other processes.

Each run records `{run_id, scenario, seed, model_start, steps_executed, card_ids}` to `evidence/scenarios/runs.jsonl`, a reproducibility log kept deliberately separate from the Historical Evidence Store (ADR-005's sole-writer invariant stays about the core evidence file, not this experiment-metadata side channel).

Occupancy truth. The Simulator owns occupancy truth directly (ADR-006): on a successful crossing it decrements the source room's count and increments the destination's, and only *then* mirrors both counts to BuildSim for visualization. A denied presentation never triggers this at all – the crossing path is gated by a bounded wait for BuildSim to report the door `UNLOCKED` (REQ-SIM-004), which a DENY decision never produces, so "an occupant never crosses a locked door" holds structurally, not by a separate check.

Why simulation is intentionally simple. The purpose of this layer is reproducible evaluation, not simulation realism: the full probabilistic `BehaviorProfile` the design specification describes (arrival/dwell-time distributions, transition probabilities) was deliberately not built. What *is* built – seeded selection, order, and timing of a small, diverse population – is exactly enough to drive every required evaluation scenario (nominal, anomalous, stress, fault) reproducibly, and no more. This is reflected honestly in `docs/traceability-matrix.md`: REQ-SIM-001/002 are PARTIAL, not PASS, for exactly this reason.

BuildSim occupancy visualization limitation. BuildSim's mirrored occupant list is not authoritative identity: when a named scenario sends several different simulated people through the door over one run, BuildSim's visualization represents the destination room's nonzero count as whichever person crossed most recently, not an accumulating roster (`internal/simulator/server.go`, `mirrorOccupancy`). The Simulator's own `/occupancy` endpoint and the `OccupancyObservation` pipeline both carry the real count regardless – only the *visual* mirror simplifies, consistent with ADR-006 stating the mirror is one-way and never read back.

How the simulator supports repeatable evaluation is the subject of Section 11: every nominal, anomalous, and stress evaluation result cited there was produced by one of these eight named scenarios, invoked with a recorded seed, and is reproducible by replaying the same `POST /scenario/run` call.

7 Autonomous decision-making and data pipeline

7.1 The autonomous service

The Decision Service combines two independently computed signals into one policy decision (Section 4.3's component diagram):

Authorization (`internal/decision/authz.go`) is a pure function implementing ADR-001's deterministic precedence chain: invalid credential → DENY; explicit user DENY → DENY; explicit user ALLOW → ALLOW; matching role DENY → DENY; matching role ALLOW → ALLOW; no match → DENY. Every step is independently unit-tested, including every pairwise conflict between a role permission and a user override.

Anomaly detection (`internal/decision/anomaly.go`) is a deterministic rule-based baseline summing two weighted factors: unusual access time (model-time hour outside 07:00–19:00, weight 0.6) and repeated attempts (the same card within 5 model seconds of its own last attempt, weight 0.6), classified `ANOMALOUS` at a summed score ≥ 0.5. It is given only the card ID and model time – never the authorization result – and its output (`anomaly_result`, `anomaly_reason`, `anomaly_score`) is recorded on every decision regardless of classification.

Decision policy combines them in exactly one place (`handleDecide`): `final_action` is set *solely* from the authorization result; the anomaly result only ever sets a separate `alert` boolean. Nothing

in the function signature or call graph gives the policy step access to override this – REQ-DEC-003 ("an anomaly score alone shall not grant access that is otherwise unauthorized") is therefore a structural property, verified directly by `TestDecide_Unauthorized_DeniedRegardlessOfAnomaly` rather than inferred from code review.

Runtime assurance is the per-door control state and relock lifecycle: an ALLOW's unlock starts a bounded no-open timer (10 model seconds, REQ-FUNC-003); a completed passage (observed via DoorStateSensor's OPEN then CLOSED) triggers an immediate relock instead. Both paths converge on the same hard constraint – a locked door never becomes unlocked except as the direct, logged consequence of an ALLOW decision.

Why the anomaly baseline is deliberately simple. Beyond the ML justification already given in Section 4.4, a third candidate factor the design specification names (unusual destination/zone) was deliberately not implemented: with exactly one controlled door, there is nothing for a destination-unusualness factor to vary against, so building it now would be untestable rather than genuinely evaluated. The two factors that *are* implemented were chosen because both have a directly constructible positive and negative test case (Section 11).

7.2 The data pipeline

The Event/Data Service (`internal/eventdata`) is the pipeline's single entry point: it stamps `receive_time`, appends the record to the evidence store, and forwards observations to the Decision Service (ADR-005). The Historical Evidence Store (`internal/evidence`) is a plain append-only JSONL file, written only by the Event/Data Service, read by the Dashboard and every evaluation tool. Every retained record type (observation, decision, command, outcome, door-state observation, occupancy observation, door event) carries a stable `event_id`/`command_id`/ `decision_id` plus `model_time`, so any record can be traced back through the chain that produced it (REQ-TEST-002, PASS). Retention is indefinite within one run (no pruning) – the file is small enough across this project's whole evaluation campaign (a few hundred kilobytes) that retention policy was never a design pressure.

8 Behaviour

8.1 Key scenario: successful passage (dynamic diagram)

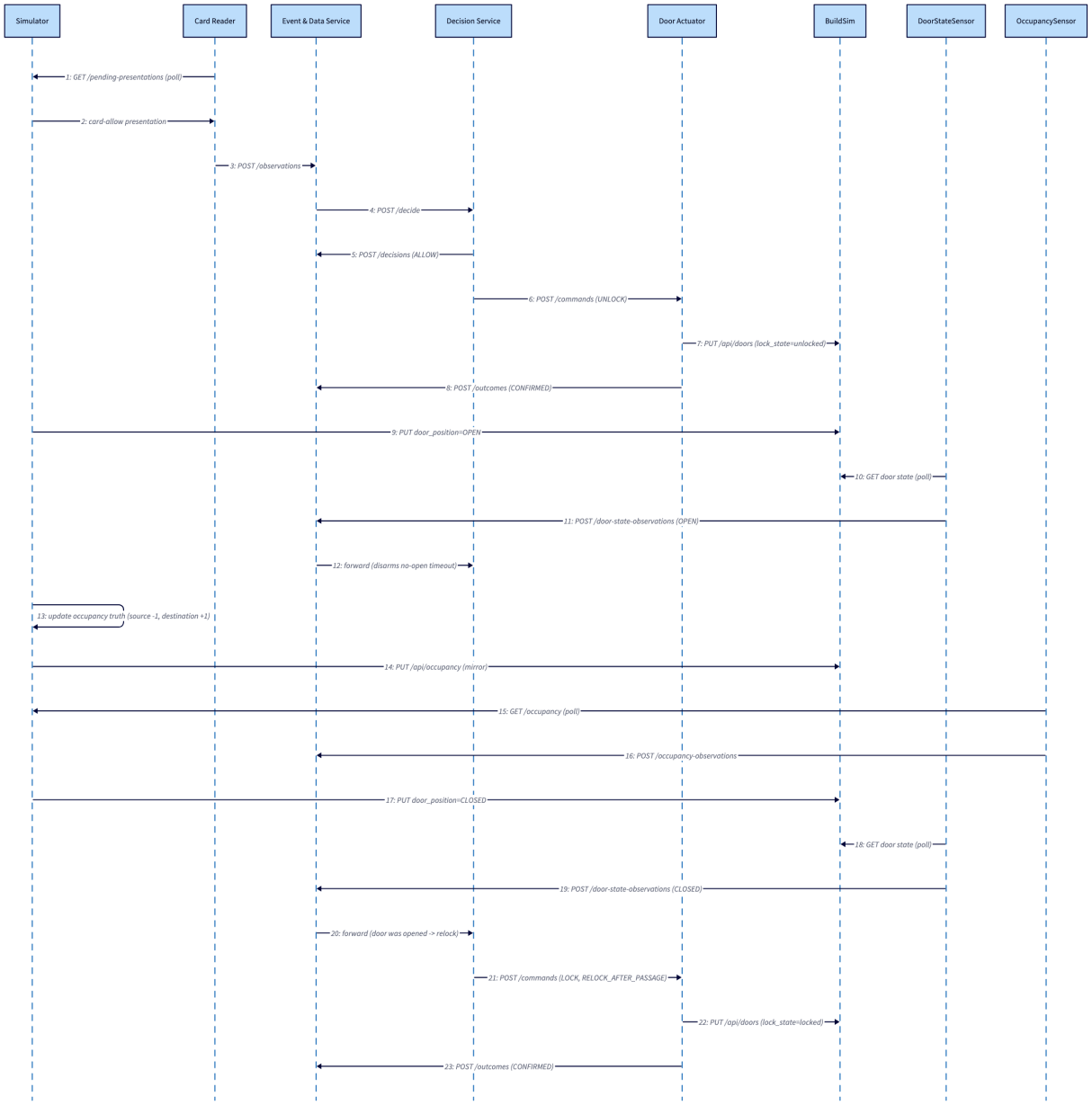


Figure 4: Dynamic diagram: the successful-passage scenario, numbered interactions among all nine runtime components.

Card Reader polls the Simulator (1–2), posts the observation to Event/Data (3), which forwards it to Decision (4). Decision records an ALLOW (5) and issues an UNLOCK to the Actuator (6), which writes to BuildSim and records a CONFIRMED outcome (7–8). The Simulator – watching for exactly this UNLOCKED state – opens the door in BuildSim (9); DoorStateSensor, polling independently, observes OPEN and forwards it, disarming the no-open timer (10–12). The Simulator then updates occupancy truth and mirrors it to BuildSim (13–14); OccupancySensor observes and records the new count (15–16). The Simulator closes the door (17); DoorStateSensor observes CLOSED (18–19); Decision, informed of the completed passage, issues a relock (20–23). Every step in this sequence is machine-checked by `test/e2e/passage_test.go` and was

reproduced against the real BuildSim binary and the Docker Compose stack during final verification (Section 9).

8.2 Door-control lifecycle (state diagram)



Figure 5: State diagram: the door-control lifecycle the Decision Service drives and reacts to.

From LOCKED, an ALLOW moves to UNLOCKED. From there, two paths exist: DoorStateSensor observes OPEN (disarming the no-open timer) and the state machine proceeds to CLOSED-then-LOCKED via `RELOCK_AFTER_PASSAGE`; or 10 model seconds elapse with no OPEN observed, producing a `DOOR_NOT_OPENED` event and an immediate LOCK via `RELOCK_TIMEOUT`. A third, orthogonal state, `UNVERIFIED`, is reachable from either LOCKED or UNLOCKED whenever a commanded transition's BuildSim confirmation fails – it is a confidence qualifier on the commanded state, not a peer physical state, and it resolves back to LOCKED or UNLOCKED once a later command succeeds. DENY never enters this machine at all: it produces no command, so the door's logical state never leaves LOCKED (REQ-FUNC-001).

9 Deployment

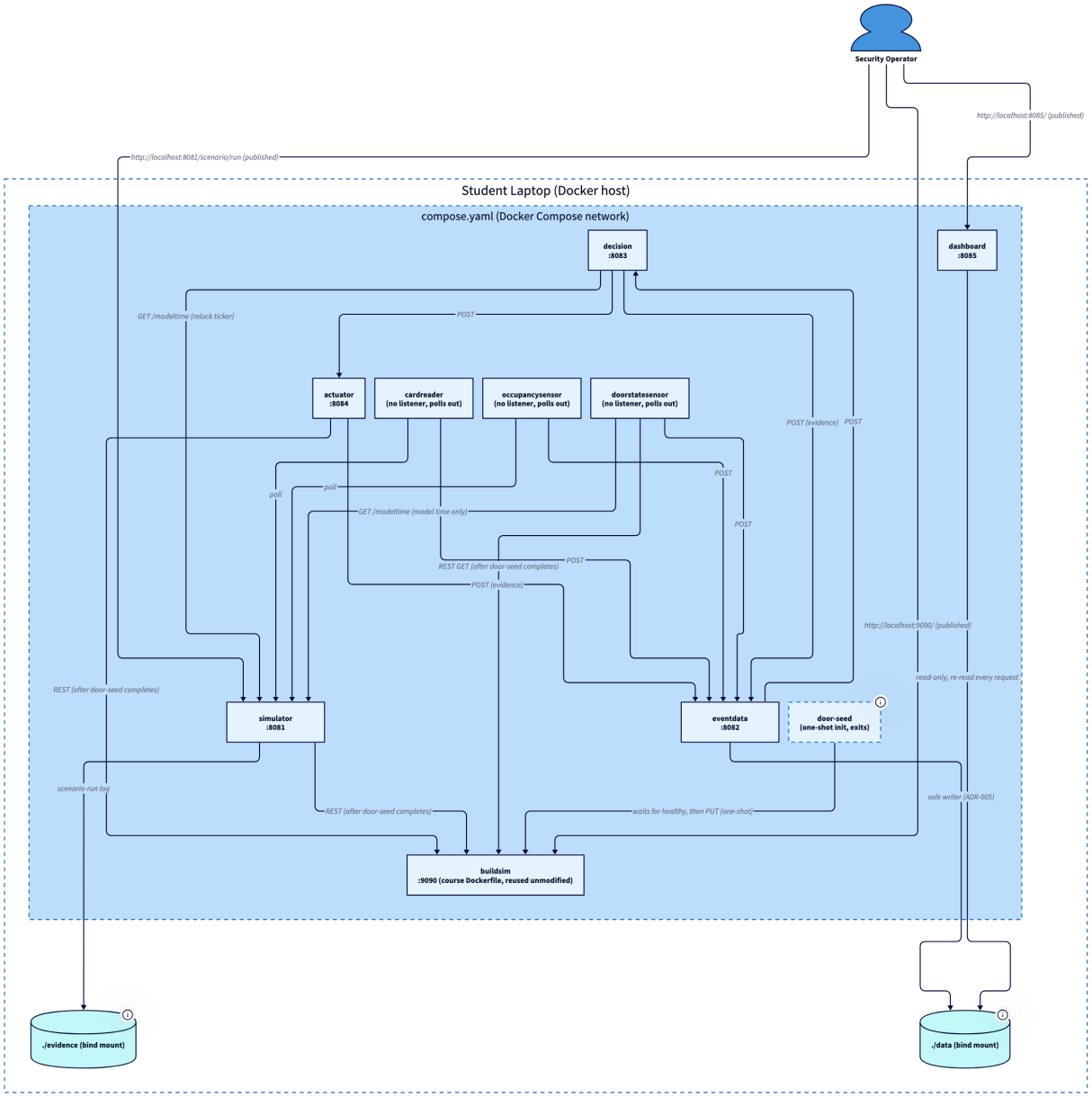


Figure 6: Deployment diagram: the Docker Compose topology – ten Compose services (nine long-running, one one-shot init step), internal DNS names, published ports, and the two persistent bind mounts.

Two supported ways to run the system exist side by side, both verified in this pass: native (`scripts/run-local.sh` + `make build/run/stop/status`, eight real Go binaries plus the real BuildSim binary on the host) and Docker Compose (`compose.yaml` at the repository root). Neither is a thin wrapper around the other – both run the exact same binaries, configured only through command-line flags.

`compose.yaml` defines ten services in total: the eight project application processes, each its own container, sharing one multi-stage `Dockerfile`-built image (`access-control:latest`) to avoid rebuilding an identical source tree eight times, each started with a different `command:`; BuildSim's own, unmodified, course-provided image (one service); and a one-shot `door-seed` init container (one service) that (re-)creates door-1 as LOCKED/CLOSED on every fresh stack

(BuildSim has no persistence, CLAUDE.md §4). Nine of the ten are expected to reach a steady running/healthy state; `door-seed` is expected to *exit* (status `0`) once door-1 is seeded, which is why `docker compose ps`'s default view – and every "nine" count elsewhere in this section – excludes it, not because it is uncounted but because it is not meant to still be running. **Independent restartability** was verified directly, not assumed: `docker compose restart decision` followed immediately by a fresh access attempt, and `docker compose stop/start actuator` around a presentation while it was down, both recovered correctly with the other eight containers continuing unaffected (`evidence/final-verification/`).

Addressing is entirely Compose-file configuration, not application code: every base-URL/listen-address flag already defaulted to `127.0.0.1/localhost` for native use; in Compose the same flags are passed Compose's internal DNS names (`buildsim`, `simulator`, `eventdata`, `decision`, `actuator`, `cardreader`, `occupancysensor`, `doorstatesensor`, `dashboard`). No `cmd/*/main.go` file needed a single change for this.

Persistent evidence. Two host bind mounts (`./data:/app/data`, `./evidence:/app/evidence`) carry `data/evidence.jsonl` and `evidence/scenarios/runs.jsonl`. Native default relative paths resolve onto them unchanged (the image's `WORKDIR` is `/app`) – no path flag override was needed either. Verified directly: both files survive a full `docker compose restart` of every container, growing normally rather than resetting.

A genuine deployment-layer defect found and fixed during this pass: a bulk `docker compose restart` (every container at once, not `up`) could race `door-seed`'s one-shot re-seed against BuildSim's own container restart, because `depends_on: condition: service_healthy` is honored by `up` but not by a bulk `restart` – observed directly as `curl: (6) Could not resolve host: buildsim`, leaving door-1 unseeded. The smallest correct fix was applied: `door-seed`'s curl call now retries (`-retry-all-errors`), which tolerates the race regardless of which restart path triggered it without changing normal startup timing. Reproduced twice before the fix, confirmed fixed twice after (`evidence/final-verification/README.md`, `docs/deployment.md` §3). This is named here deliberately, not only in the risks section, because it is exactly the kind of change-and-failure behavior worth demonstrating with evidence rather than asserting.

10 Test plan and verification

Level	Coverage
Unit tests	`internal/decision` (authorization precedence – 13 tests, anomaly scorer in isolation – 6 tests, full policy matrix – 3 tests), `internal/actuator` (confirmation, deduplication, BuildSim-unavailable – 4 tests), `internal/modeltime` (format/ parse/advance determinism – 4 tests), `internal/simulator` (seed reproducibility – 1 test).
End-to-end tests	`test/e2e/`: ALLOW, DENY, relock-timeout, successful-passage (`FakeBuildSimClient`), decision fail-closed/restart/recover, and five named-scenario tests exercising the real `POST /scenario/run` interface.
Integration test	`test/integration/realbuildsim_test.go`: the same ALLOW/relock sequence against the real, course-provided BuildSim binary, not the fake.
Scenario tests	Authorized crossing, denied access, after-hours anomaly, rapid-repeat anomaly, burst count – all via the production `POST /scenario/run` interface, not a test-only backdoor.
Docker Compose verification	A full manual pass against the real containerized stack: all services healthy, door-1 seeded, all four scenario types, a container restart, an actuator-stop fault, dashboard reachability, and evidence persistence – repeated at the end of this pass with a fresh build (`evidence/final-verification/`).

Results, this pass (2026-10-10): `gofmt -l .` clean; `go vet ./...` clean; `go test ./...` and `-race` green across every package; `go vet -tags=integration ./...` clean; `go test`

`-tags=integration ./test/integration/...` green against the real BuildSim binary; all three `config/*.json` files valid JSON; `docker compose config/build/up` all succeed; all nine long-running Compose services reach running/healthy (the tenth, `door-seed`, exits as expected – Section 9). Full logs, exact commands, and timestamps: `evidence/final-verification/`.

Requirement traceability is `docs/traceability-matrix.md` (Section 3): every one of the 54 internal requirements and all 9 mandatory course pass/fail criteria are mapped to a specific test or experiment and an evidence location there, not merely asserted.

11 Evaluation

This section is evidence, not assertion: every number below is read from a committed evidence file (`docs/evaluation-results.md` is the same content in prose-and-table form with full source citations).

11.1 Nominal correctness

Every functional scenario in `docs/scenarios.md` was exercised and produced the expected result: ALLOW/DENY, the full successful-passage sequence, the no-open relock timeout, and reproducible seeded selection of a population subset (`normal-day`, seed 42, consistently selecting `[card-henry, card-frank, card-dave, card-erin, card-carol]` across repeated runs). See `docs/evaluation-results.md` §1.

11.2 Latency (REQ-PERF-001)

$n = 200$ presentations, in-process over loopback (`cmd/latencybench`), wall-clock.

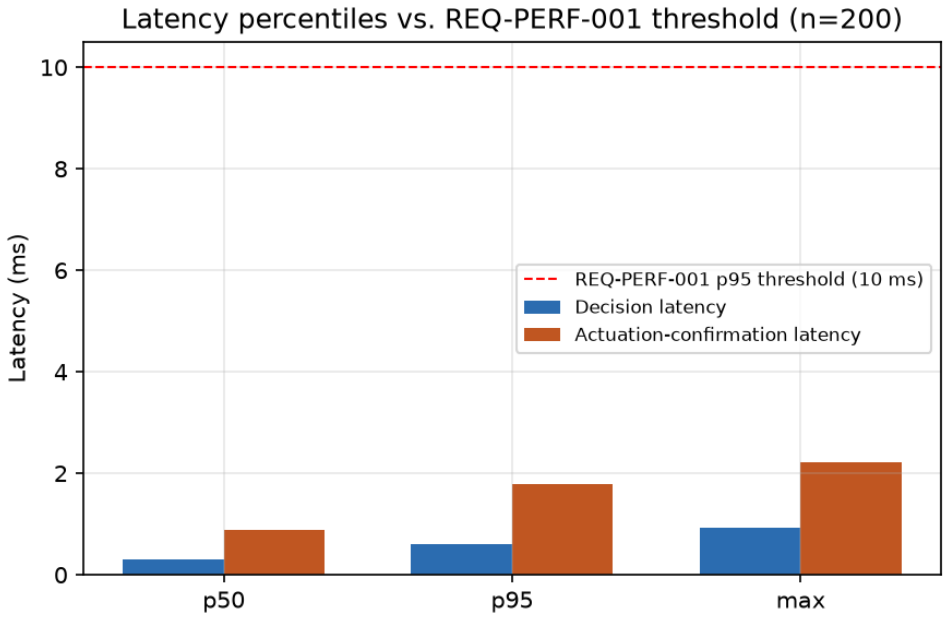


Figure 7: Decision and actuation-confirmation latency percentiles against the REQ-PERF-001 threshold.

Metric	p50	p95	max	Threshold
Decision latency (ms)	0.298	0.601	0.928	p95 $\leq$ 10 ms – met

Metric	p50	p95	max	Threshold
Actuation-confirmation latency (ms)	0.884	1.791	2.21	no formal threshold; secondary metric

The threshold (p95 $\leq$ 10 ms) was set *after* this baseline was measured, deliberately – high enough above the measured 0.601 ms to absorb scheduling jitter and modest load growth, but far below the ∼100 ms point at which an interactive response is perceived as non-instantaneous, so it remains a real engineering constraint rather than one loosened until it passed. **Caveat, stated plainly**: both numbers were measured against `FakeBuildSimClient`, not the real BuildSim HTTP adapter's GET-modify-PUT round trip – the actuation path's true latency against the real adapter was not separately benchmarked.

11.3 Failure / recovery



Figure 8: Decision Service restart recovery time across 10 fresh runs.

Killing the Decision Service mid-attempt produces no decision and no command – a locked door stays locked (REQ-RES-001) – confirmed both automatically and against a real killed OS process. Restarting it (a fresh process, no carried-over state) recovers correctly; 10 runs captured during this pass measured **min 10.83 ms, mean 18.8 ms, max 32.26 ms** from restart to the first correct ALLOW/UNLOCK. Killing the dependency the Actuator needs (BuildSim itself, or the Actuator container) produces a recorded `UNLOCK` command whose outcome is `UNVERIFIED` with an empty `observed_state` – never a false `CONFIRMED` (REQ-RES-002) – and recovers fully once the dependency returns.

11.4 Anomaly policy behaviour

Scenario	Authorization	Anomaly	Final action	Alert
Normal authorized (day)	ALLOW/ ROLE_ALLOW	NORMAL	ALLOW	false
Anomalous but authorized (night)	ALLOW/ ROLE_ALLOW	ANOMALOUS/ UNUSUAL_ACCESS_TIME	ALLOW	true

Scenario	Authorization	Anomaly	Final action	Alert
Unauthorized (also night)	DENY/ NO_PERMISSION	ANOMALOUS/ UNUSUAL_ACCESS_TIME	DENY	true
Rapid repeat, 2nd/3rd attempt	ALLOW/ ROLE_ALLOW	ANOMALOUS/ REPEATED_ATTEMPT	ALLOW	true

Row 3 is the direct proof of REQ-DEC-003: the anomaly classification is reported honestly regardless, but it never changes `final_action` – a DENY stays a DENY even when it is also, separately, anomalous.

11.5 Stress / breaking-point experiment

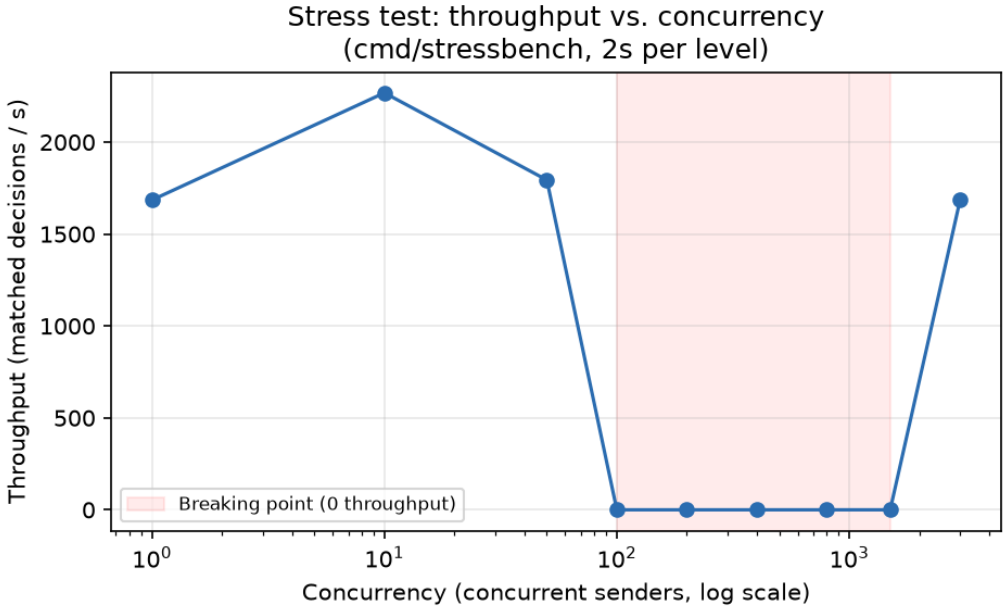


Figure 9: Throughput collapses to zero between concurrency 50 and 100, partially recovering at 3000.

`cmd/stressbench`, concurrency swept 1–3000, firing directly at `POST /observations` (bypassing the Card Reader's own single-poll-loop bottleneck deliberately, to measure the pipeline's own limit, not that unrelated one).

Concurrency	Throughput (req/s)	p95 (ms)	Err.	Interpretation
1	1685	0.20	0%	Nominal
10	2268	1.86	0%	Nominal
50	1794	4.50	3.1%	Early degradation
100	**0**	–	**100%**	**Breaking point**
200–1500	0	–	100%	Total failure persists
3000	1687	1877.77	72.9%	Partial, unstable recovery

Root cause, confirmed not assumed: the Decision Service's own log shows `dial tcp ...: can't assign requested address` on its outbound call to the Door Actuator. Every inter-service call shares `http.DefaultClient`, whose default transport allows only 2 idle connections per host; each access attempt fans out into ∼4 internal HTTP calls, so at concurrency ∼100 the

system has hundreds of simultaneous short-lived loopback connections competing for a two-deep pool per link. This was confirmed by first ruling out a benchmark-tool artifact (undrained response bodies on the first sweep): after fixing the tool and re-running, the ceiling did not move – the error originates in the production Decision Service's own log, not the benchmark client. **Not optimized in this pass** – the purpose was to find and honestly report the limit, not move it (Section 13).

A second, previously-unknown gap this experiment surfaced: when Decision→Actuator delivery itself fails (distinct from Actuator→BuildSim failing, which REQ-RES-002 already covers), a command is recorded as issued with **no outcome ever following** – worse than `UNVERIFIED`, which is at least a record that something was attempted. Not fixed; named here and in Section 13.

12 Dashboard

The dashboard (`internal/dashboard`, `cmd/dashboard`) reads the evidence file directly and re-reads it fresh on every request – no separate API, no cached/stale view. It shows: the controlled door's `lock_state` and `door_position` with a verification indicator (CONFIRMED vs. UNVERIFIED); the latest access decision, including `authorization_reason`, `anomaly_result/reason`, and `alert`; destination-room occupancy; a per-source-ID "component health" panel derived purely from each source's most recent evidence timestamp (explicitly *not* a real heartbeat protocol – stated as such in the code, not oversold); and a table of recent evidence records. It auto-refreshes every two seconds via a plain `<meta http-equiv="refresh">`, deliberately avoiding a WebSocket/SSE layer the system has no other use for. Reachable at `http://localhost:8085/` both natively and via Docker Compose, confirmed live during final verification.

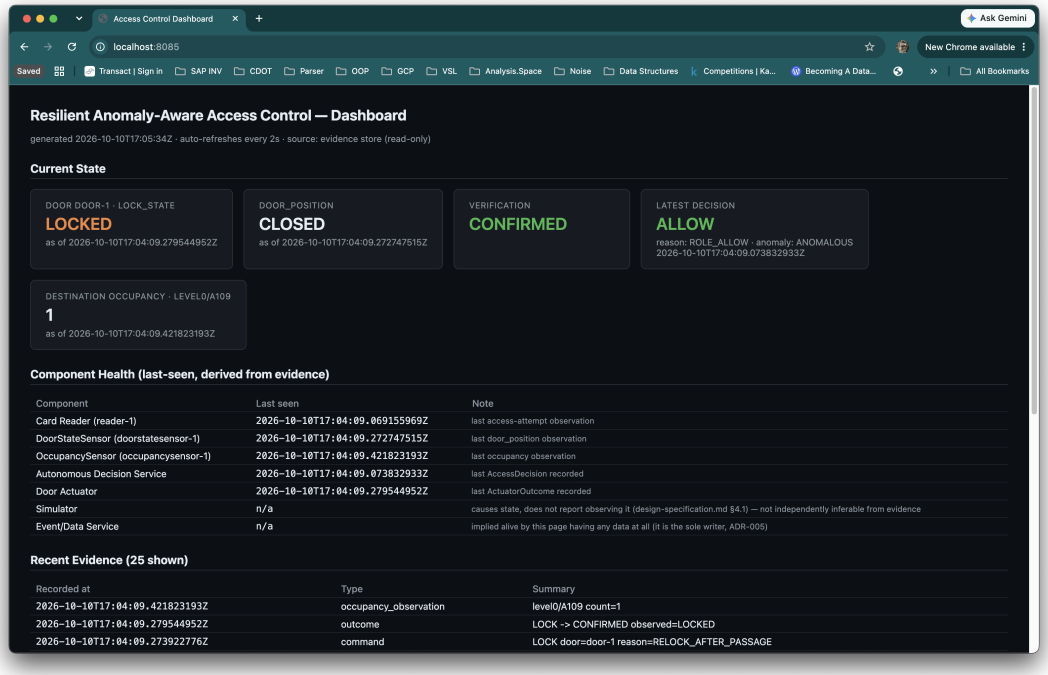


Figure 10: Operational dashboard showing live door state, recent access decisions, anomaly status, occupancy, and retained evidence.

13 Risks, limitations, and future work

An honest account, not a polished one – naming a real limitation is worth more here than claiming there are none.

- **One controlled, representative door.** The entire architecture (reader/actuator/door-state binding, zones, topology) generalizes to many doors without a redesign (REQ-TOPO-002/003 are already topology-generic), but only one was ever built or tested. What would change at 100 doors or multiple buildings: the Event/Data Service's single evidence file would need partitioning or a real store (it was never meant to scale past this project's evidence volume); the Decision Service's per-door control state (already a map keyed by door ID) would need to handle many more concurrent entries under real contention, not just the occasional overlapping presentation found during this pass (below); and BuildSim's own wholesale-replace PUT semantics would become a genuine throughput concern at that scale, not just an adapter-design curiosity.
- **7 of 54 internal requirements are unimplemented**, consistently named across this project's whole development, not discovered late: device re-registration after restart (REQ-DIST-005/006 – no `/api/equipment` flow exists); and data-quality fault handling (REQ-AUTH-005 time windows, REQ-DATA-004/005 duplicate/late detection, REQ-RES-003/005 staleness). 6 more are PARTIAL. Full list and rationale: `docs/traceability-matrix.md`.
- **Deterministic anomaly baseline, not a learned model** – justified in Section 7, but it means the system cannot adapt to behavior patterns not anticipated by its two fixed factors, and its thresholds (0.6/0.6/0.5, the time window, the repeat window) are a defensible starting point, not derived from a real evaluation dataset – none exists for this project.
- **No probabilistic crowd simulation.** Section 6 states this as a deliberate scope choice, not an oversight: REQ-SIM-001/002 are PARTIAL for exactly this reason.
- **BuildSim's occupancy mirror is not authoritative identity** – Section 6's visualization limitation; the Simulator's own count and the `OccupancyObservation` pipeline remain correct regardless.
- **BuildSim is a shared dependency across three independent processes** (Actuator, Simulator, DoorStateSensor) – a single point of failure for all three at once, named explicitly when discovered (`evidence/actuator-failure/README.md`) rather than worked around silently.
- **Connection-pool stress bottleneck**, confirmed root cause, deliberately not optimized (Section 11) – the obvious first fix (a larger `internal/httpx` connection pool) is named, not built, because this pass's purpose was to find and report the limit honestly, not move it.
- **Decision→Actuator missing-outcome failure mode**, discovered by the stress experiment, not fixed: a command can be recorded with no resolution at all if delivery to the Actuator itself fails.
- **Single-pending-per-door control state can be clobbered by overlapping presentations.** Discovered during Docker verification when a rapid-repeat scenario's trailing presentations overlapped a fresh restart-recovery attempt on the same door: the per-door bookkeeping (keyed by `door_id`, not `decision_id`) can lose track of an earlier unlock's relock obligation. A clean, isolated request immediately afterward completed correctly, confirming this is a pre-existing property of the single-slot design, not a restart-recovery regression. Not fixed – redesigning per-door state to be per-decision or queued is out of scope for this pass.
- **Only selected failure domains were tested.** Decision Service and the Actuator↔BuildSim

link were fault-tested explicitly; Card Reader/OccupancySensor/DoorStateSensor crash-recovery is architecturally identical (same restart pattern, same "unrelated processes unaffected" property) but was not separately exercised, to avoid repeating the same proof three more times for diminishing evidentiary value.

- **Actuation-confirmation latency caveat**: measured against `FakeBuildSimClient`, not the real BuildSim adapter's GET-modify-PUT round trip (Section 11).
- **Deferred stale/late/duplicate observation handling** (REQ-DATA-004/005, REQ-RES-003/005) – no freshness, ordering, or deduplication logic exists anywhere in the pipeline yet.
- **No BuildSim device/equipment re-registration flow** (REQ-DIST-005/006) – the real adapter talks to `/api/doors` directly; a restarted sensor/actuator process resumes polling/commanding immediately, but there is no explicit registration handshake to lose or repeat.
- **A genuine deployment defect was found and fixed, not just documented**, within the scope the task allowed for a small, safe correction: the `door-seed` vs. bulk-restart race (Section 9).

If more time existed, data-quality fault handling (duplicate/late/malformed detection in the Event/Data Service) is the single highest-value remaining item: it is explicitly named in the grading criteria's Dimension D examples, cheap relative to full behavioral-profile simulation (no new process, just validation logic in an already-existing service), and would produce new, easy-to-demonstrate evidence using the exact same capture pattern already established four times over in this project (`docs/final-report-gap-analysis.md`).

14 Conclusion

The original question was whether an autonomous edge system can make safe building-access decisions from imperfect sensor data while remaining robust to communication and component failures. Within the tested prototype envelope, the answer is **yes, with explicitly identified limits**.

Safe: every safety invariant CLAUDE.md names was verified, not merely asserted – an invalid or disabled card never unlocks a door; a DENY never produces an unlock command; the anomaly score, tested directly, cannot override a denial; a failed Decision Service leaves a locked door locked; an unconfirmed actuator result is recorded `UNVERIFIED`, never a false `CONFIRMED`; duplicate commands produce no duplicate physical effect.

Robust to failure, within what was tested: a killed Decision Service recovers in tens of milliseconds with no carried-over state; a killed BuildSim/Actuator dependency produces an honestly-unverified record and recovers fully once restored; every one of the eight project containers restarts independently in Docker without disturbing the others; and a genuine deployment-layer race condition, found by this pass's own verification rather than assumed away, was fixed and reverified rather than merely written down.

Within explicit limits: the system was stress-tested to an actual breaking point, not just a nominal-load check, and that limit's root cause was confirmed, not guessed at. A real, previously-unknown gap (Decision→Actuator delivery failing silently into a resolution-less command) was discovered by that same stress experiment and is reported rather than concealed. Seven of 54 internal requirements remain unimplemented and six more partial, all named consistently across this project's whole development rather than only at the end. The anomaly baseline is deliberately simple and would not generalize past its two fixed factors without further work.

None of these limits were papered over to make this report read better than the system behaves. What they show, collectively, is a system whose behavior, failure modes, and boundaries can be explained from evidence – which is the actual claim this report makes, not that the system is complete.